

Assignment 1: Observability

Enterprise Software Development, Fall 2026

This is an individual assignment. Dates and submission details will be announced separately.

You will add metrics and logging to an application of your choice, then use them to investigate problems. You may use [Lab 1: Midnight Launch](#) as a reference.

Part A: Your project

Choose a problem and build a working solution. Your project can be as simple or complex as you like.

State the problem, intended users, and your solution. Explain what works and how to try it.

Part B: Metrics

Set up Prometheus and Grafana. Track both:

- Application metrics, such as response times and failed requests.
- Business metrics, such as orders placed and cancellations.

Use all four metric types and show their data in Grafana. For a food-ordering app, these could be:

Type	Simple example
Counter	Orders placed so far: 20 → 21 when someone places an order.
Gauge	Orders waiting to be prepared: goes up when an order arrives, down when it is ready.
Histogram	Count order requests taking up to 0.1 seconds, up to 0.5 seconds, and up to 1 second.
Summary	Report average order-request time and, if supported, p95: "About 95% of requests took 0.8 seconds or less."

Show p95 or p99 and explain the time window. [Histograms can provide percentiles too](#). [Python summaries lack percentiles](#): show their average, and use a histogram for p95/p99.

List your metrics in a table. For each, give its purpose, type, unit, labels, and where and how it is recorded in the code. Include the Grafana query and explain what the chart shows.

Cover the important parts of your app and business, not just class examples. Include at least one metric or chart you explored yourself. You need all four types across the project, not four for each category.

Add [Node Exporter](#) to measure CPU, memory, disk, and network usage. Collect these with Prometheus and display them in Grafana. Name the machine being measured.

Part C: Logs

Set up Filebeat, Elasticsearch, and Kibana. A basic pipeline looks like this:

App writes a JSON log → Docker saves it → Filebeat reads and parses it
→ Elasticsearch stores it → You search it in Kibana

In your report, explain:

- What you log, why, and where in the code.
- How Filebeat collects logs and turns them into searchable fields.
- Where logs live, what survives restarts, and when old logs are deleted.
- How you search for a request or error in Kibana.

Include time, service, severity, message, and request ID where relevant. Show one original log, its stored fields, and a working search. Explain parsing if using plain text. Never log secrets or personal data. Logstash is optional.

Part D: System design

1. Architecture diagram

Draw your app, dependencies, and monitoring tools. Add arrows and notes showing:

- What each component does and how they communicate.
- Where data is stored and why you chose this design.
- What happens if a component stops working.

You should be able to explain how the tools work together and why you chose this setup. You are not expected to know every line of their source code. Note anything you do not yet understand.

2. Follow a metric and a log

Take one metric and one log from your app and walk through their paths:

- Metric: code updates it → Prometheus collects and stores it → Grafana queries and displays it.
- Log: code writes it → Filebeat collects and parses it → Elasticsearch stores it → Kibana finds it.

Show values, queries, and changes to the log's format.

Part E: Experiments

1. Reproduce a problem

Choose slow requests, failures, or another problem relevant to your app.

1. Run a repeatable test and show normal behavior.
2. Predict which metrics and logs will change.
3. Introduce the problem and show what actually changes.
4. Explain the cause and the effect on users.
5. Remove the problem and repeat the test to check recovery.

For example, add a 500 ms delay to every fifth request. Compare response-time charts and logs before, during, and after the delay.

Include the commands, time ranges, queries, and results. Run each stage long enough for several Prometheus scrapes. Test locally and make sure you can undo the fault.

2. Cardinality explosion

Cardinality explosion happens when labels with many different values, such as request IDs, create too many time series. See the [Prometheus label guidance](#).

1. Add a `request_id` label to a local test counter. Stop at 100 unique IDs.
2. Show series growth, e.g. `count(demo_requests_total)` .
3. Remove the label, restart the test app, and repeat. Compare after another scrape.
4. Explain the cost at larger scale and why IDs belong in logs.

Do not try to crash Prometheus. Removing labels does not immediately delete stored history.

What to submit

- Code and configuration, including Grafana dashboards and Kibana queries.
- A README with instructions to start, use, test, and safely clean up your project.
- A short report with sections A–E, including your metric list, diagrams, explanations, and results.

You do not need slides. Credit reused code, sources, and assistance according to course policy. Exclude downloaded dependencies and data volumes.

Proposed marks

Part	Marks
A — Working project and clear problem	10
B — Metrics and dashboards	30
C — Logging pipeline	25
D — System design and explanation	20
E — Anomaly (10) and cardinality experiment (5)	15
Total	100

Marks are based on whether your setup works and whether you can explain it using your results, not on the size of your project.